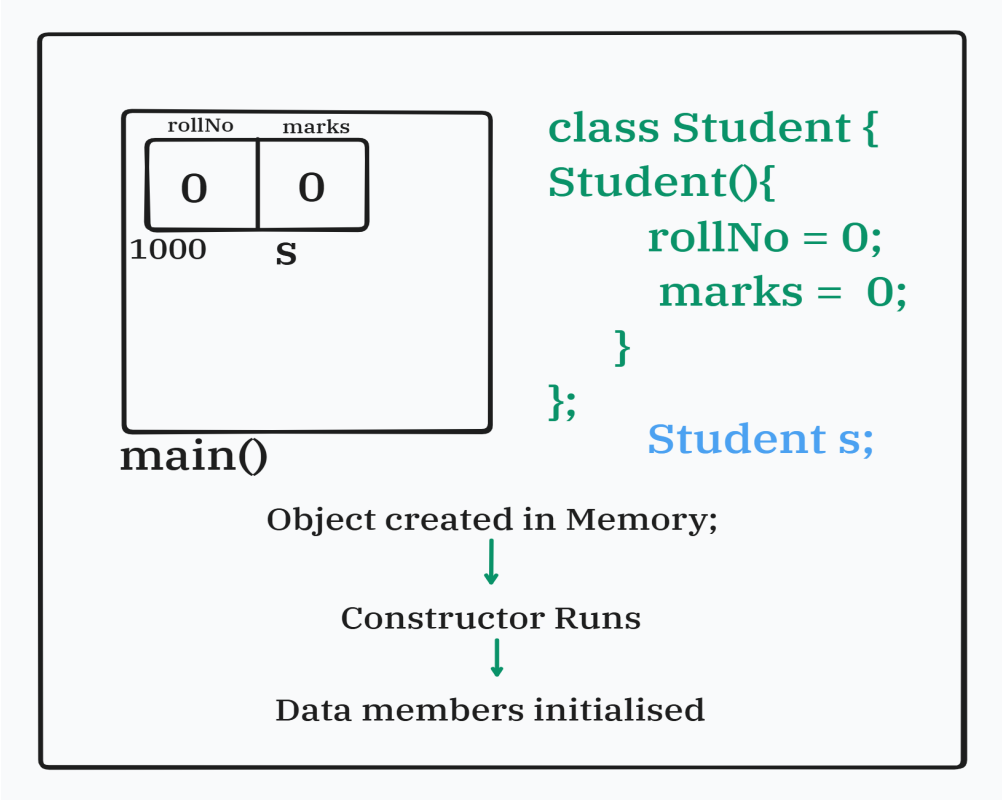
27/2/25 DAY 5

Constructor concept

- A constructor is a special member function of a class that is automatically called when an object of the class is created.
- It is used to initialize the object's data members and set up any necessary resources.
- The constructor has the same name as the class and does not have a return type, not even void.
- Constructors can be overloaded, meaning a class can have multiple constructors with different parameters to provide various ways of initializing objects.
- If no constructor is explicitly defined in a class, the compiler provides a default constructor. This does not garantuee that the data members will be initialized to any specific values; they may contain garbage values if not explicitly initialized.
- The main purpose of a constructor is to ensure that an object is in a valid state when it is created, and to provide a convenient way to set initial values for the object's data members.



Default constructor

- A default constructor is a constructor that can be called with no arguments.
- It is either provided by the compiler if no constructors are defined, or it can be explicitly defined by the programmer.
- Syntax for default constructor:

```
class ClassName {
public:
    ClassName() {
        // constructor body
    }
};
```

```
#include <iostream>

using namespace std;

class Student{
    int rollNo;
    float marks;

public:
    //default constructor
    Student(){
        rollNo = 0;
        marks = 0;
    }
    float getMarks() const {
        return marks;
    }

    void setMarks(float m) {
        marks = m;
    }

    int getRollNo() const {
        return rollNo;
    }

    void setRollNo(int rn) {
        rollNo = rn;
    }
};

int main(){
    Student s;//default constructor called

    cout << "Student RollNo: "<< s.getRollNo() << " Student Marks: "<<
s.getMarks() << endl;
    return 0;
}
```

Parameterized constructor

- A parameterized constructor is a constructor that takes one or more parameters to initialize the object's data members with specific values at the time of object creation.
- It allows for more flexible and customized object initialization compared to a default constructor.
- Syntax for parameterized constructor:

```
class ClassName {
public:
    ClassName(parameter_list) {
        // constructor body
    }
};
```

```
#include <iostream>

using namespace std;

class Student{
    int rollNo;
    float marks;

public:

    // parameterized constructor
    Student(int rn, float m){
        rollNo = rn;
        marks = m;
    }

    float getMarks() const {
        return marks;
    }

    void setMarks(float m) {
        marks = m;
    }

    int getRollNo() const {
        return rollNo;
    }

    void setRollNo(int rn) {
        rollNo = rn;
    }
};

int main(){
    Student s1(101, 50);// Parameterized constructor called

    s1.setRollNo(121);
```

```
    s1.setMarks(60);

    cout << "Student RollNo: "<< s1.getRollNo() << " Student Marks: "<<
s1.getMarks() << endl;

    return 0;
}
```

Multiple constructors

- A class can have multiple constructors with different parameter lists, allowing for various ways to initialize objects of the class.
- This is known as constructor overloading. The appropriate constructor is called based on the arguments provided during object creation.

```
#include <iostream>

using namespace std;

class Student{
    int rollNo;
    float marks;

public:
    //default constructor
    Student(){
        rollNo = 0;
        marks = 0;
    }

    // parameterized constructor with rollNo only
    Student(int roll){
        rollNo = roll;
    }

    // parameterized constructor with marks only
    Student(float m){
        marks = m;
    }

    // parameterized constructor
    Student(int rn, float m){
        rollNo = rn;
        marks = m;
    }

    float getMarks() const {
        return marks;
    }

    void setMarks(float m) {
```

```
        marks = m;
    }

    int getRollNo() const {
        return rollNo;
    }

    void setRollNo(int rn) {
        rollNo = rn;
    }
};

int main(){

    Student s;              //default cons
    Student s1(101, 50);    // Parameterized cons
    Student s2(55.5);       // single parameter cons

    s1.setRollNo(121);
    s1.setMarks(60);

    cout << "Student RollNo: "<< s.getRollNo() << " Student Marks: "<<
s.getMarks() << endl;
    cout << "Student RollNo: "<< s1.getRollNo() << " Student Marks: "<<
s1.getMarks() << endl;
    cout << "Student RollNo: "<< s2.getRollNo() << " Student Marks: "<<
s2.getMarks() << endl;

    return 0;
}
```

Default arguments

- Default arguments allow you to specify default values for parameters in a constructor. If an argument is not provided during object creation, the default value will be used.
- This can help reduce the number of overloaded constructors needed in a class.
- This can be used in member functions as well.
- Syntax for default arguments in constructor:

```
class ClassName {
public:
    ClassName(parameter_list = default_value) {
        // constructor body
    }
};

```c++
#include <iostream>
using namespace std;

class Student{
```
```

```
    int rollNo;
    float marks;

public:
    // right to left
    Student(int rn = 0, float m = 0){
        rollNo = rn;
        marks = m;
    }

    // this is valid
    // Student(int rn , float m = 0){
    //  rollNo = rn;
    //  marks = m;
    // }

    // this is not valid: default arguments should be from right to left
    // Student(int rn = 0, float m){
    //     rollNo = rn;
    // }

    float getMarks() const {
        return marks;
    }

    void setMarks(float marks) {
        this->marks = marks;
    }

    int getRollNo() const {
        return rollNo;
    }

    void setRollNo(int rollNo) {
        this->rollNo = rollNo;
    }
};

int main() {
//  Student s;
    Student s1(101, 50);

//  cout << s.rollNo << " " << s.marks<<endl;
cout << s1.rollNo << " " << s1.marks<<endl;
    return 0;
}
```

this pointer

- The `this` pointer is an implicit pointer available in all non-static member functions of a class.
- It points to the current object for which the member function is called.

- It is used to access the members of the current object, especially when there is a naming conflict between member variables and parameters or local variables.
- It can also be used to return the current object from a member function, enabling method chaining.

```
class ClassName {
public:

    ClassName(parameter_list) {
        // constructor body
    }

    //compiler will convert this to:
    // ClassName(parameter_list, ClassName *this)

    void memberFunction() {
        // function body
    }
    //compiler will convert this to:
    // void memberFunction(ClassName *this)
};

int main() {
    ClassName obj; // object creation
    obj.memberFunction(); // member function call
    // compiler will convert this to:
    // memberFunction(&obj);
    return 0;
}
```

```
#include <iostream>
using namespace std;

class Student{
    int rollNo;
    float marks;

public:

    // this pointer Student const *this
    // in case of const object: const Student const *this
    Student(int rollNo = 0, float Marks = 0){
        this->rollNo = rollNo;
        this->marks = Marks;
    }

    float getMarks() const {
        return marks;
    }

    void setMarks(float marks) {
```

```
        this->marks = marks;
    }

    int getRollNo() const {
        return rollNo;
    }

    void setRollNo(int rollNo) {
        this->rollNo = rollNo;
    }
};


int main() {
    Student s;  // this -> object address
    const Student s1(101, 50);

//  const Student s2(100);

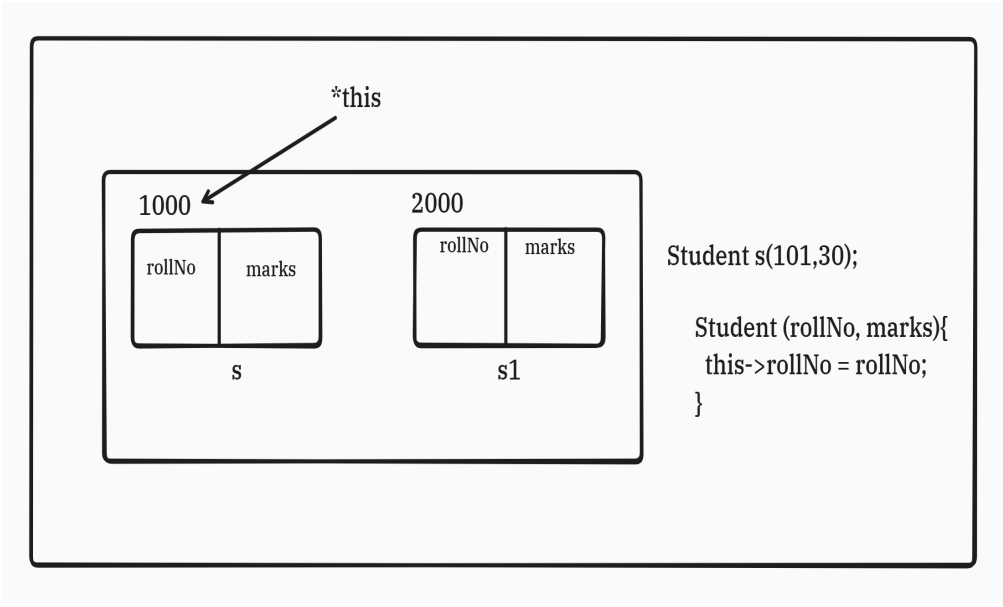
    //error: passing 'const Student' as 'this' argument discards qualifiers
//  s1.setRollNo(111);
//  s1.setMarks(20);

//  setRollNo(&s, 101);
//
//  void setRollNo(Student *this, int rollNo){
//      this->rollNo = rollNo;
//  }(

    s.getRollNo(); // s1 - x, s ==

    cout << "Roll NO: " << s.getRollNo() << " Marks: " << s.getMarks() << endl;
    cout << "Roll NO: " << s1.getRollNo() << " Marks: " << s1.getMarks() << endl;

    return 0;
}
```



const data members

- A const data member is a member variable of a class that cannot be modified after it has been initialized.
- It must be initialized at the time of declaration or in the constructor initializer list, as it cannot be assigned a value in the constructor body.
- The const qualifier ensures that the value of the data member remains constant throughout the lifetime of the object.

```
class ClassName {
public:
    ClassName(parameter_list) : constMember(value) {
        // constructor body
    }
};
```

Constructor initializer list

- A constructor initializer list is a syntax in C++ that allows you to initialize member variables of a class before the constructor body executes.
- It is more efficient than assigning values to member variables inside the constructor body, especially for const members and reference members, which must be initialized at the time of declaration.
- The issue with initializing const data members in the constructor body is that they cannot be assigned a value after they have been initialized. Therefore, they must be initialized using an initializer list.
- The syntax for a constructor initializer list is as follows:

```
ClassName(parameter_list) : member1(value1), member2(value2), ... {
    // constructor body
}
```

```
#include <iostream>
using namespace std;

class Student{
    const int rollNo;
    float marks;

public:
    //para cons
    Student(int rollNo, float marks) : rollNo(rollNo), marks(marks){

    }

    // const Student* this const
    float getMarks() const {
        return marks;
```

```
    }

    void setMarks(float marks) {
        this->marks = marks;
    }

    int getRollNo() const {
        return rollNo;
    }
};

int main() {

    Student s(101, 50);
    Student s1(101, 54);

    cout << s.getRollNo() << " | " << s.getMarks() << endl;
    cout << s1.getRollNo() << " | " << s1.getMarks() << endl;

    return 0;
}
```

const member functions

- A const member function is a member function that cannot modify any non-mutable member variables of the class.
- It is declared with the `const` keyword after the function parameters.
- The `const` keyword is part of the function signature and allows the function to be called on const objects.

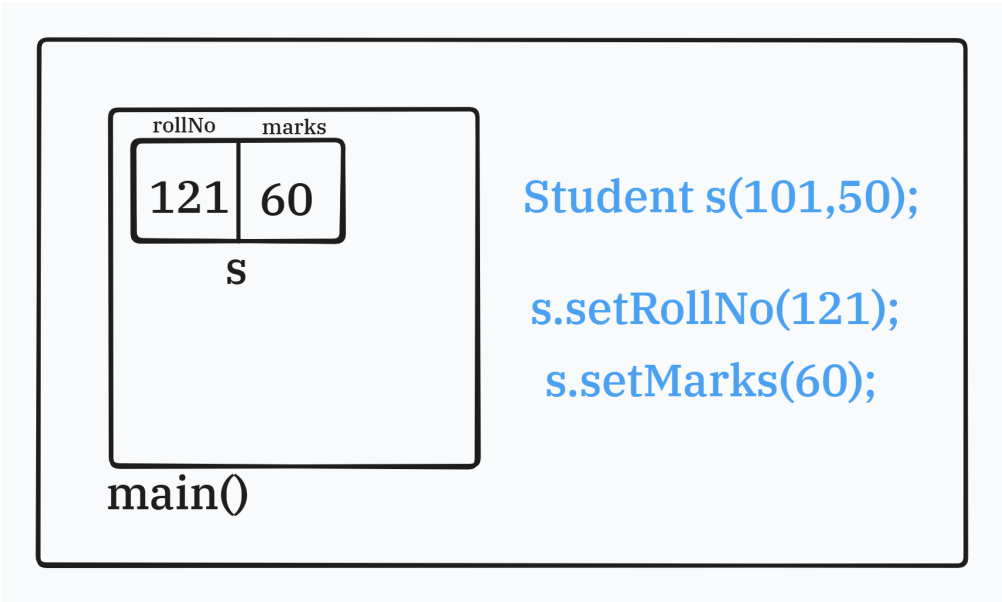
```
class ClassName {
public:
    // const member function
    // compiler will convert this to:
    // void constMemberFunction(const ClassName const *this)
    // this pointer will be of type const ClassName* because the function is const
    void constMemberFunction() const {
        // function body
    }

    // compiler will convert this to:
    // void nonConstMemberFunction(ClassName const *this)
    void nonConstMemberFunction() {
        // function body
    }
};
```

Object memory layout

- The memory layout of an object in C++ typically consists of the following components:

1. **Data members**: These are the variables that hold the state of the object. They are stored in contiguous memory locations within the object.
2. **Padding**: To ensure proper alignment of data members, the compiler may insert padding bytes between them. This is done to optimize memory access and improve performance.



Object lifecycle (stack vs heap objects)

STACK

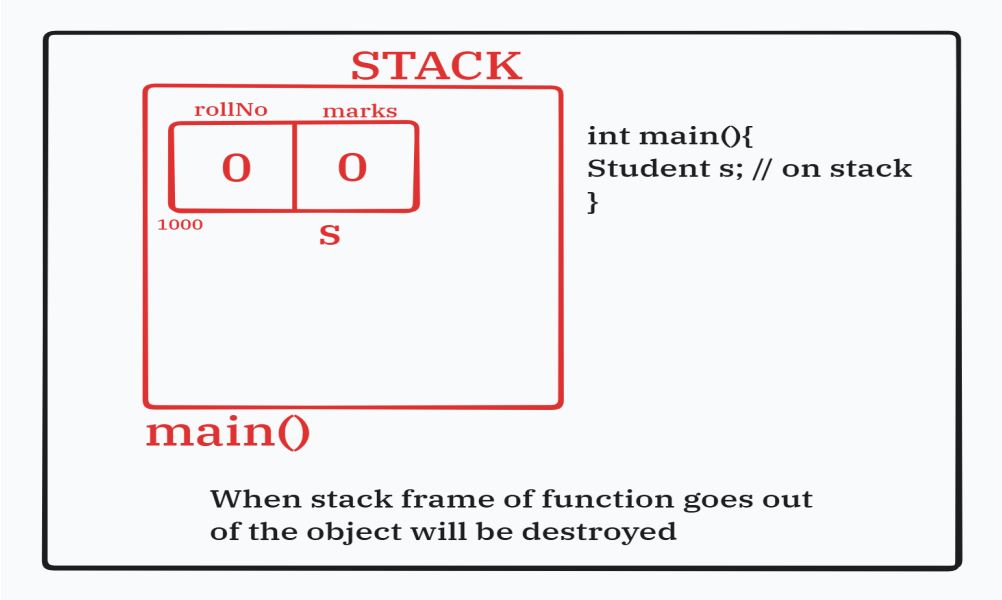
- Stack memory is used for local variables and function call management. Objects created on the stack are automatically destroyed when they go out of scope.

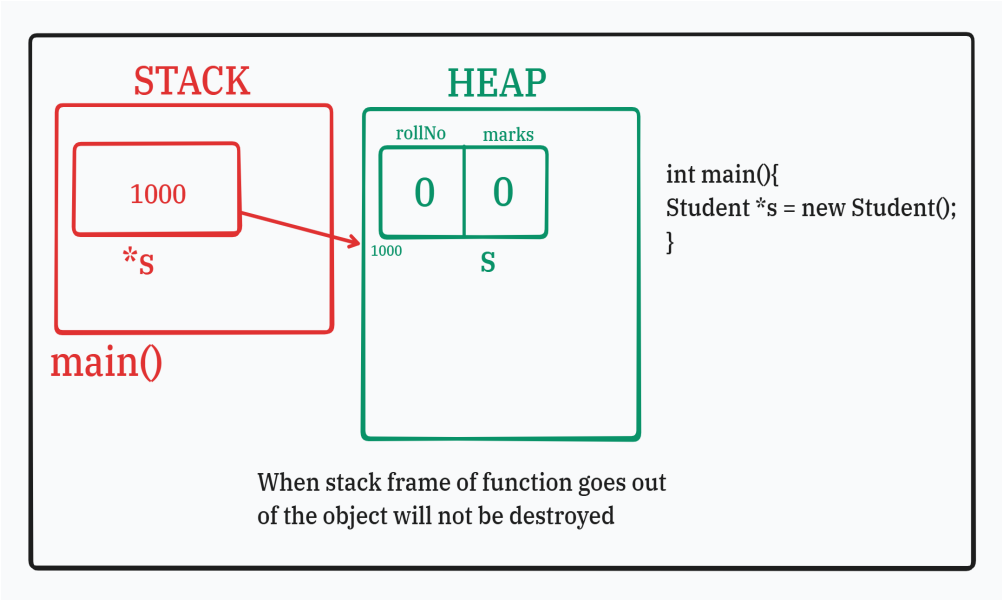
```
Object will be created -> It will get memory -> initialized with constructor ->
scope end -> destroyed
```

HEAP

- Heap memory is used for dynamic memory allocation. Objects created on the heap must be explicitly destroyed by the programmer using the `delete` operator to free the allocated memory.

```
Object will be created -> It will get memory -> initialized with constructor ->
scope end -> explicitely destruction
```


STACK
rollNo
marks
0
0
1000
s
main()
int main(){
Student s; // on stack
}
When stack frame of function goes out
of the object will be destroyed


STACK
HEAP
1000
*s
main()
rollNo
marks
0
0
1000
s
int main(){
Student *s = new Student();
}
When stack frame of function goes out
of the object will not be destroyed